

Phishing detection model

Julian Calvo. AAI6850, Module 15 capstone. August 2026.

This is the model that gets sold on the NexusAI marketplace. Its evaluation output is what the oracle publishes, and the marketplace refuses to sell it if the published accuracy drops below the floor the provider declared.

1. Problem

Binary classification of email as phishing or benign, from the subject, body, sender headers and any URLs in the body.

The costs are asymmetric. A false negative lets a phishing email into an inbox. A false positive quarantines legitimate business mail, and enough of those and the security team turns the filter off. I weighted a false negative at 4x a false positive and required precision of at least 0.90 at whatever operating point I picked.

2. Related work

Phishing detection splits into four lines of work. They solve different problems, even though the papers get compared against each other as if they didn't.

Hand-crafted features on the email itself. Fette, Sadeh and Tomasic built PILFER (WWW 2007) on ten features: IP-address URLs, the age of the linked domain, anchor text that doesn't match its href, HTML mail, link and domain counts, dots in a URL, JavaScript, and the SpamAssassin score. A ten-tree random forest over those reported 99.5% accuracy at a 0.13% false positive rate, on roughly 860 phishing messages against 6,950 SpamAssassin ham messages. The argument of the paper is that a small set of structural features beats a general spam filter on this task.

Content and search-engine signals on the landing page. CANTINA (Zhang, Hong and Cranor, WWW 2007) takes the five highest TF-IDF terms on a page, adds the domain, and searches for them. If the page's own domain isn't in the results, the page is treated as phishing. On 100 phishing and 100 legitimate pages that reported 97% true positive at 6% false positive. Adding heuristics moved it to 89% TP at 1% FP, which is the trade they were after. CANTINA+ (Xiang, Hong, Rosé and Cranor, TISSEC 2011) expanded to 15 URL, DOM and search features behind a login-form filter, feeding a Bayesian network, and reported 92.54% TP at 0.407% FP on unique phishing pages. The 99% figure often quoted from that paper is on near-duplicate phish a hash filter already catches, not on novel ones.

URL only, with no page fetch. Ma, Saul, Savage and Voelker (KDD 2009) classify a URL from lexical tokens plus host data (WHOIS, DNS, AS, geolocation), 30,000 features or more, with L1-regularised logistic regression, reporting 0.9% to 3.0% error. PhishDef (Le, Markopoulou and Faloutsos, INFOCOM 2011) drops the host lookups, uses lexical features only, and learns online with AROW, reporting 96% to 98% accuracy, about a point behind the full feature set. Neither has to fetch anything, so they are fast and hard to block.

Learned text representations. Fang et al. (IEEE Access 2019) model header and body at both character and word level through an RCNN with attention, reporting 99.848% accuracy at 0.043% FPR. Atawneh and

Aljehani (Electronics 2023) compare CNN, RNN, LSTM and BERT, with a BERT plus LSTM hybrid reporting 99.61% accuracy and 99.55% F1.

Method	Year	Classifies	Features	Model	Reported
PILFER	2007	email	10 structural, plus SpamAssassin score	random forest	99.5% acc, 0.13% FP
CANTINA	2007	web page	TF-IDF signature plus search result	linear heuristics	97% TP, 6% FP
CANTINA+	2011	web page	15 URL, DOM and search	Bayesian network	92.5% TP, 0.41% FP
Ma et al.	2009	URL	lexical plus WHOIS, DNS, AS	L1 logistic regression	0.9 to 3.0% error
PhishDef	2011	URL	lexical only	AROW, online	96 to 98% acc
Fang et al.	2019	email	char and word, header and body	RCNN with attention	99.85% acc, 0.043% FP
Atawneh, Aljehani	2023	email	tokenised text	BERT plus LSTM	99.61% acc, 99.55% F1
This project	2026	email	text, URL and header	soft-vote ensemble	96.0% F1, synthetic corpus

Those numbers are not comparable and the table should not be read as a ranking. The task differs: PILFER and the deep learning papers classify emails, CANTINA classifies rendered pages, Ma and PhishDef classify bare URLs. The corpora differ in source, era and class balance, from roughly one phish per eight ham in PILFER to balanced in Ma and PhishDef. Atawneh and Aljehani don't state their class balance at all, so their 99.61% has no denominator a reader can check. False positive rate at a stated operating point survives the comparison better than accuracy does, which is why section 6 reports mine that way.

What I took from this. PILFER's result is the one my model ladder reproduces. Model M3 throws the email text away, keeps only URL and header features, and beats every text model I tried. PILFER reached that conclusion in 2007 on completely different data. The deep learning results are the argument for the future work in section 12, since a transformer over the body is the obvious thing my TF-IDF block leaves on the table. I didn't attempt one here, partly because a 99.8% number on a corpus I generated myself would mean nothing.

3. Data

The corpus is synthetic. The machine I built this on can't reach the public phishing datasets, so `ml/data/build_dataset.py` generates records from a seeded RNG (seed 6850) using 114 templates. Loaders for Nazario, SpamAssassin, PhishTank and the UCI phishing dataset are written and get tried first. They have never run against real files here.

	Records	Phishing	Benign	Groups
Corpus	8,000	3,356	4,644	1,574
Holdout	1,200	481	719	805

The holdout includes 300 records from templates that appear nowhere in the corpus. 1.5% label noise is injected deliberately, so a perfect score is not achievable and anything close to one would be a bug.

Leakage control

Templates produce near-duplicate messages. A random train/test split would put siblings of a test message in training and report a score that means nothing. So the split is `StratifiedGroupKFold` on `group_id`, where a group is a near-duplicate cluster.

Split sizes: train 4,797, val 1,603, test 1,600. Group overlap between splits is zero, checked and asserted.

I measured what the grouping is worth by running the identical pipeline both ways:

Split	Test F1
Random	0.9411
Grouped	0.9358

That probe uses a fixed logistic regression pipeline rather than the selected ensemble, so those numbers aren't comparable to the 0.960 in section 7.

So a random split would have overstated F1 by about 0.005. Small here, because the synthesiser spreads templates fairly evenly. On real corpora with heavy campaign duplication it would be much larger, which is why the control stays in.

4. Features

Three families, all built in `ml/src/features.py`.

A. Text. TF-IDF over subject and body. Word 1-2 grams, plus `char_wb` 3-5 grams. The character n-grams are there because deliberate misspellings like "account" and "verify", and homoglyph domains, shatter into unseen tokens under word tokenisation but stay recognisable at character level.

B. URL and lexical, 30 features. Count of URLs, HTTPS ratio, maximum and mean URL entropy, longest URL, risky TLDs, shortener hosts, brand lookalike distance, subject and body length, punctuation counts.

C. Header and authentication, 22 features. SPF, DKIM and DMARC results, sender domain shape (digits, hyphens, length, depth), freemail sender, reply-to present, reply-to mismatch, reply-to freemail, display-name brand mismatch, received hop count, role account.

5. Model ladder

Each step tests one idea rather than sweeping hyperparameters. Validation F1 at threshold 0.5:

Model	What changed	Val F1
M1	Baseline. Word TF-IDF + logistic regression	0.7516
M2	Add char n-grams	0.7830
M3	Drop text entirely, URL + header features only, HistGradientBoosting	0.9705
M4	All three families, calibrated LinearSVC	0.9359
M5	All three families, logistic regression, grouped GridSearchCV	0.9463
M6	Soft vote over M3 and M5	0.9682

M3 is the result I did not expect. Throwing away the text entirely and using only engineered URL and header features beats every text model by a wide margin. Most of the signal in this corpus lives in who sent it and where the links point. Section 2 covers why that lines up with the older literature.

M4 doing worse than M3 is the text block dragging down a model that already had the useful features.

M6 is selected, on validation PR-AUC (0.9753). M3 and M5 fail differently: M3 can't read the text at all, M5 can't model feature interactions like "SPF fails AND reply-to mismatches". Averaging their probabilities trims both error modes.

6. Operating point

Selected threshold 0.2723, chosen to minimise expected cost (FN weighted 4x) subject to precision staying above 0.90.

Threshold	Precision	Recall	F1	Cost
0.272 (selected)	0.9715	0.9657	0.9686	111
0.386 (F1-optimal)	0.9831	0.9553	0.9690	131
0.500 (default)	0.9846	0.9523	0.9682	138

F1 hardly moves across that range but cost moves 25%, so I picked the threshold on cost. F1 wouldn't have distinguished them.

7. Test results

Held out until the model and threshold were both frozen. Evaluated once.

Metric	At 0.272	At 0.5
Accuracy	0.9663	0.9719
Precision	0.9572	0.9816
Recall	0.9629	0.9510
F1	0.9600	0.9660
ROC-AUC	0.9813	0.9813
PR-AUC	0.9740	0.9740

Test F1 0.960 against validation 0.969. A small drop is what you want to see. A test score above validation would suggest the split leaked.

Plots are in `ml/reports/`: ROC and PR curves, confusion matrix, feature importance.

8. Error analysis

29 false positives and 25 false negatives on the test set. Of those, 21 FPs and 9 FNs are genuine model failures. The rest land on records whose label the noise process deliberately flipped, so the model was right and the label was wrong.

Full worked examples are in `ml/reports/error_examples.md`.

False negatives. 12 of the 25 pass all three authentication checks. That's the pattern: a phishing message sent from a genuinely authenticated domain defeats the strongest feature family the model has. One has no URLs at all, which removes the second family too, leaving only the text the model has learned to under-weight.

False positives. Almost all are legitimate mail with no URLs. With no links to score, the URL family contributes only zero-valued features, and several of those zeros push toward phishing (`https_ratio = 0` is the largest single contributor on most of them). The model treats "no links" as suspicious because in this corpus phishing messages more often have no scoreable links.

What I'd do about it. Add an explicit "no URLs present" indicator so the model can distinguish "no links" from "bad links", rather than encoding both as zero. And weight the header family less when authentication passes, since a passing SPF says less than it used to now that attackers can get one.

9. Feature importance

Top drivers toward phishing, from the dense model:

`sender_domain_digits`, `sender_is_freemail`, `sender_domain_risky_tld`, `display_brand_mismatch`, `reply_to_freemail`, `any_risky_tld`, `url_max_entropy`.

Top drivers toward benign: `display_has_brand`, `brand_lookalike_min_dist`, `https_ratio`, `any_brand_lookalike`, `reply_to_present`.

Permutation importance on M3 agrees: `sender_is_freemail`, `sender_domain_digits`, `reply_to_freemail`, `received_hops`, `brand_lookalike_min_dist`.

The text features that surface are mostly synthesiser artifacts. `time_sensitive`, `kind_regards` and `dropbox` are template phrases rather than phishing signal.

10. Ethics

False positives. Quarantined business mail costs real time and, in the wrong context (a legal deadline, a customer escalation), real money. Worse, a noisy filter gets ignored, which makes the false negatives worse too. That's why precision has a floor and isn't just traded off against recall.

False negatives. They are what the model exists to prevent, and why the cost model weights them 4x.

Privacy. The model reads email content, which is about as sensitive as data gets. In deployment it should run inside the buyer's boundary, not as a service that sees their mail. Nothing in this pipeline ships email content anywhere, and nothing about a buyer or their mail goes on chain. The oracle publishes an accuracy number and a manifest hash, nothing else.

Evasion. An attacker who knows the features can defeat them: get a real domain with valid SPF, avoid digits and hyphens, use plausible display names. The header family is the strongest and also the most gameable. Anyone deploying this should assume it degrades under adversarial pressure and re-evaluate on a schedule. The oracle's staleness window forces that anyway.

Bias. The corpus is English-only, from a synthesiser I wrote, so it inherits my assumptions about what phishing looks like. Non-English mail, unusual business conventions, and legitimate senders on freemail

domains are all likely to be treated unfairly. Freemail is a top phishing feature, and small businesses legitimately send from freemail addresses.

11. Limitations

1. The corpus is synthetic. This is the big one.
2. English only.
3. No attachment analysis, no HTML structure, no image content.
4. No temporal split. Real phishing evolves and a chronological holdout would be a harder and more honest test.
5. The authentication features assume headers you can trust, which in practice means you're inside the receiving infrastructure.
6. The threshold was picked on one validation split, not bootstrapped, so there is no confidence interval on it.

12. Future work

Re-run on Nazario, SpamAssassin and PhishTank. Add a temporal split. Try a transformer on the text block and see whether it beats the dense features (I'd guess it helps most on the cases where headers are clean). Add the no-URL indicator from section 7. Bootstrap the threshold. Publish the holdout manifest so a third party can reproduce the oracle score, which the payload already carries the hash for.

13. References

1. Fette, Sadeh and Tomasic, Learning to detect phishing emails, WWW 2007 (PILFER)
2. Zhang, Hong and Cranor, CANTINA: a content-based approach to detecting phishing web sites, WWW 2007
3. Xiang, Hong, Rosé and Cranor, CANTINA+: a feature-rich machine learning framework for detecting phishing web sites, ACM TISSEC 14(2), 2011
4. Ma, Saul, Savage and Voelker, Beyond blacklists: learning to detect malicious web sites from suspicious URLs, KDD 2009
5. Le, Markopoulou and Faloutsos, PhishDef: URL names say it all, IEEE INFOCOM 2011
6. Fang, Zhang, Huang, Liu and Yang, Phishing email detection using improved RCNN model with multilevel vectors and attention mechanism, IEEE Access 7, 2019
7. Atawneh and Aljehani, Phishing email detection model using deep learning, Electronics 12(20), 2023
8. Pedregosa et al., scikit-learn: machine learning in Python, JMLR 2011